ISA 401: Business Intelligence & Data Visualization

10: Web Scraping I

Fadel M. Megahed, PhD

Raymond E. Glos Professor in Business
Farmer School of Business
Miami University

@Fadel Megahed
fmegahed
fmegahed@miamioh.edu
Automated Scheduler for Office Hours

Fall 2026

Before We Start

1 Join today's Zoom

Use the Zoom link in the **Week 0 Module** on Canvas. **Mute** your mic and **turn off** your video.
Log in from the **lab's computer** so you can **share your screen** if you need help with an error.

2 Open the project, start the markdown

GitHub Desktop → `business_intelligence` → **Fetch origin**, then open `business_intelligence.Rproj`.
New R Markdown, title *Class 10*, delete everything below the setup chunk, save it as **markdowns/class10.Rmd**.

3 Install two packages

In the **Console** (not in the markdown): `install.packages(c("rvest", "robotstxt"))`
`httr2` is already installed from Class 08.

4 Optional, on your own laptop

Add the **SelectorGadget** extension to Chrome. It is one of two ways to find a selector today; the other, the **Inspector**, is built into every browser.

Done early? **Help the person next to you.** Stuck? **Share your screen on Zoom** and we fix it together.

Where We Left Off

Class 08: an API

The owner **documents** an address and its parameters. The reply is **JSON written for programs**: only data, in named fields. Some APIs ask for a key.

Today: a web page

No documentation and no key. The reply is **HTML written for people**: the data sits inside tags, next to menus, links, and images.

The first course on Miami's ISA catalog page, as the server sends it

```
<div class="courseblock">
<p class="courseblocktitle"><strong>ISA 125.  Introduction to Business Statistics.  (3)</strong></p>
<p class="courseblockdesc">
This course provides an introduction to data, probability, sampling and its importance to analytical decision-making in business. Upon
successful completion of this course, students will have the foundational skills necessary to summarize data, describe relationships among
variables, and conduct one-sample and two-sample statistical inference. Note: Credit for graduation will not be given for more than ...
```

When an API exists, use it. When the data exists only on a page, we read the page's HTML and pull out the parts we need. That is web scraping, and it starts with checking that we are allowed to.

Scraping in Job Ads

Where Job Scout's 1,794 postings came from



■ Employer career sites, via Active Jobs DB ■ USAJOBS API (Class 08)

The 1,342 career-site postings link to **575 different sites**; 375 of them are pages hosted by Workday.

Job Scout buys this feed; it does not scrape. The feed's maker, Fantastic.jobs, publishes a tool called "Jobs Scraper for ATS / Career Sites" and describes its data as coming from "200k+ career sites and 54 ATS platforms". Postings harvested Jul 29 to Aug 23, 2026, the weeks both sources returned postings (`data/scout.db`).

A skill some jobs ask for: three examples from Job Scout

Business Analyst, Hisense USA
Alpharetta, GA. Posted Jul 23, 2026.
*"Apply Python-based data analysis techniques, including **web scraping tools**, to collect and monitor pricing data across multiple sales channels."*

AI Automation & Business Analytics Intern, Brio Water Technology
Glendale, CA. Posted Jul 16, 2026.
*"Develop or improve **web scrapers** and data collection tools when needed."*

Data Science Intern, Mendoza Research Team, University of Notre Dame
Notre Dame, IN. Posted Aug 18, 2026.
*"Clean, merge, and validate complex research datasets, including financial/market data, administrative records, survey results, and **web-scraped data**."*

Two sides of the same skill. Most of these postings reached Job Scout because someone scraped employer career sites, and some employers hire analysts to do the same work: track prices, collect data, and clean what was scraped.

Learning Objectives for Today's Class

- **Determine** when scraping is permitted (robots.txt, terms of service).
- **Parse** HTML using `rvest`.
- **Extract** text, tables, and attributes from HTML elements.

Part 1 14 min	**Part 2** 18 min	**Part 3** 35 min
When can we scrape? robots.txt, terms of service, and one court case.	**How a web page is built.** HTML elements, CSS selectors, and two tools that find a selector for you.	**Scraping with rvest.** The workflow, the User-Agent header, all the tables on a Wikipedia page, then Miami's course catalog and a practice job board.

Wednesday: many pages at once with `purrr::map()`, polite delays between requests, and pages where some fields are missing.

Part 1: When Can We Scrape?

robots.txt

A plain text file at the root of a site, written for programs that visit pages automatically. **User-agent** names the program (***** means every program); **Disallow** lists paths they should not visit.

bulletin.miamioh.edu/robots.txt (excerpt)

```
User-agent: *
Disallow: /archive/
Disallow: /admin/
Disallow: /azindex/
Disallow: /catalogcontents/
...
```

The course pages under `/courses-instruction/` are not on the list.

www.linkedin.com/robots.txt (first lines, then line 4,857)

```
# Notice: The use of robots or other automated means to access LinkedIn
without
# the express permission of LinkedIn is strictly prohibited.
# See https://www.linkedin.com/legal/user-agreement.
...
User-agent: *
Disallow: /
```

`Disallow: /` covers every path on the site. Named crawlers, such as Googlebot and Bingbot, get their own rules higher in the file.

Question. May a program read the ISA page of Miami's course catalog?

```
robotstxt::paths_allowed(paths = "/courses-instruction/isa/", domain = "bulletin.miamioh.edu", bot = "*")
```

Returns **TRUE**. robots.txt is a **request** from the site owner, not a law. Respecting it is the first thing a careful scraper does.

Check Three Sites

03:00

Task Answer Discussion

In `class10.Rmd`, use `robotstxt::paths_allowed()` to check each path for every program (`bot = "*"`):

Site	Path
`en.wikipedia.org`	`/wiki/Miami_RedHawks_football`
`www.linkedin.com`	`/jobs/`
`www.indeed.com`	`/jobs` and `/viewjob`

Then open one of the robots.txt files in the browser and find the line that decided your answer.

Terms of Service

LinkedIn User Agreement, section 8.2 "Don'ts"

"Develop, support or use software, devices, scripts, robots or any other means or processes (such as crawlers, browser plugins and add-ons or any other technology) to **scrape or copy the Services, including profiles and other data** *from the Services"*

LinkedIn User Agreement, effective Nov 3, 2025. Signing up for an account means accepting it.

Wikipedia's copyright page

"Permission is granted to copy, distribute and/or modify Wikipedia's text under the terms of the **Creative Commons Attribution-ShareAlike 4.0** *International License"*

Wikipedia:Copyrights. Reuse requires credit, such as a link to the page.

robots.txt says which paths programs should visit. **The terms of service** say what you may do with the site and its data, and they bind you once you accept them (for example, by creating an account).

Look for the terms in the page footer ("Terms", "User Agreement", "Legal") and search them for *scrape*, *crawl*, *robot*, and *automated*.

hiQ Labs v. LinkedIn

hiQ collected public LinkedIn profiles and sold analyses of them to employers. LinkedIn sent a cease-and-desist letter and blocked it; hiQ sued.

Aug 2017
District court orders LinkedIn to let hiQ keep collecting public profiles while the case goes on.

Sep 9, 2019
Ninth Circuit upholds the order: scraping public pages is likely not access "without authorization" under the federal anti-hacking law (CFAA).
Opinion

Jun 14, 2021
Supreme Court sends the case back to be reconsidered in light of a new CFAA decision, *Van Buren*.
TechCrunch

Apr 18, 2022
Ninth Circuit reaches the same conclusion again.
Opinion

Nov 4, 2022
District court: hiQ **breached LinkedIn's User Agreement**, by scraping and by using fake profiles.
Order

Dec 2022
Consent judgment: **$500,000** against hiQ; hiQ stops scraping LinkedIn and destroys the scraped data and code built on it.
Privacy World

What the case shows. The court did not treat scraping public pages as hacking, and hiQ still lost, because it had agreed to LinkedIn's terms and then broken them. The terms of service are part of the permission check.

The 2017 order is described in the 2019 opinion. This is a U.S. case; other countries have their own rules, and this slide is not legal advice.

Part 2: How a Web Page Is Built

One Page, Two Views

In the browser: what a reader sees



View page source (Ctrl+U): what the server sent

```
<div id="textcontainer" class="page_content">
<div id="sc_sccoursedescs">
<div class="courseblock">
<p class="courseblocktitle"><strong>ISA 125.  Introduction to Business
Statistics.  (3)</strong></p>
<p class="courseblockdesc">
This course provides an introduction to data, probability, sampling and its
importance to analytical decision-making in business. Upon successful
completion of this course, students will have the foundational skills
necessary to summarize data, describe relationships among variables, and
conduct one-sample and two-sample statistical inference. Note: Credit for
graduation will not be given for more than one of STA 125, <a
href="/search/?P=ISA%20125" title="ISA 125" class="bubblelink code"
onclick="return showCourse(this, 'ISA 125');">ISA 125</a>, <a
href="/search/?P=STA%20261" title="STA 261" class="bubblelink code"
onclick="return showCourse(this, 'STA 261');">STA 261</a>, <a
href="/search/?P=STA%20301" title="STA 301" class="bubblelink code"
onclick="return showCourse(this, 'STA 301');">STA 301</a>, or STA 368.
<br>Prerequisites: MTH 102 or <a href="/search/?P=MTH%20121" title="MTH 121"
class="bubblelink code" onclick="return showCourse(this, 'MTH
121');">MTH 121</a> or <a href="/search/?P=MTH%20125" title ...
```

The same course, twice. The browser reads the tags to decide how each piece looks; **rvest reads the same tags to decide which pieces to keep.**

Miami University General Bulletin 2026-2027, Information Systems & Analytics (ISA). Screenshot taken Sep 28, 2026.

HTML Elements

One element from the Wikipedia bowl games table (two attributes left out)

```
<a href="https://en.wikipedia.org/wiki/1948_Sun_Bowl" title
="1948 Sun Bowl">Sun Bowl</a>
```

Tag: a, a link. The start tag `<a ...>` and end tag `</a>` wrap the content.

Attribute name: `href`, where the link goes. `title` is a second attribute.

Attribute value: the address, inside quotes. A reader never sees it.

Content: the text a reader sees on the page.

> Common tags: `h1` to `h6` headings, `p` paragraph, `a` link, `img` image, `div` and `span` containers, `table`, `tr` row, `th` header cell, `td` data cell.

Elements sit inside elements

```
<html>
  <head>  title, styles: not shown
  <body>  everything on the page
    ...
    <table class="wikitable">
      <tbody>
        <tr>  one per row
          <th> Season </th> ...
        <tr>
          <td> <a>1947</a> </td>
          <td> <a>Sun Bowl</a> </td> ...
```

This nesting is the **tree** that rvest searches. The `class` attribute on the table is how we will point at it.

CSS Selectors

`.parent`	every element with `class="parent"`
`div.parent`	every with class parent
`.child.rebel`	elements with both classes, child and rebel
`.parent .rebel`	class rebel inside an element with class parent
`#p1`	the element with `id="p1"`

#p1 matches **1** element. # selects by **id**. An id is unique on a page, so there is at most one match.

A class attribute can hold several names separated by spaces: `class="child rebel"` is class child **and** class rebel.

```
<h1>This is a sample html</h1>

<blockquote>
<p>Maybe stories are just data with a soul.</p>
<footer>Brene Brown</footer>
</blockquote>

<div id="p1" class="parent">
Hmm
<p>Hi!</p>
How are you?
<div class="child nice">
  <p>Hello!</p>
</div>
</div>

<p>Household 1</p>

<div class="parent">
<p>Hi!</p>
<blockquote class="child rebel">
  <p>Don't talk to me!</p>
</blockquote>
</div>

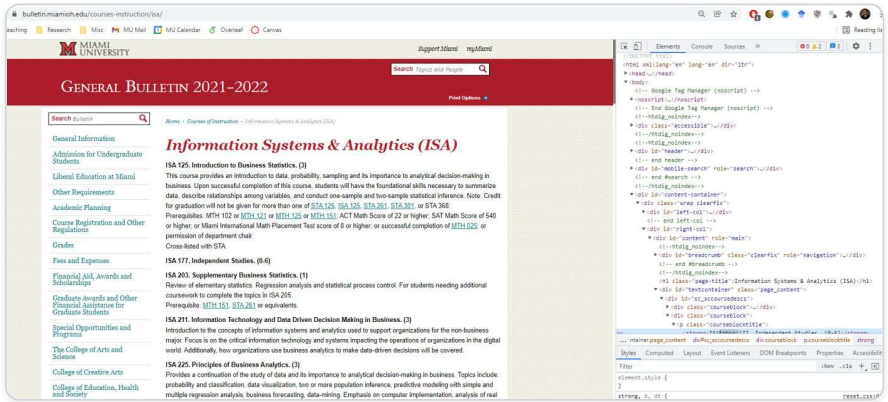
<span class="child">
<span class="parent child rebel">
  <p>Clean your room!</p>
</span>
</span>

<p>End of households</p>
```

Two Tools for Finding a Selector

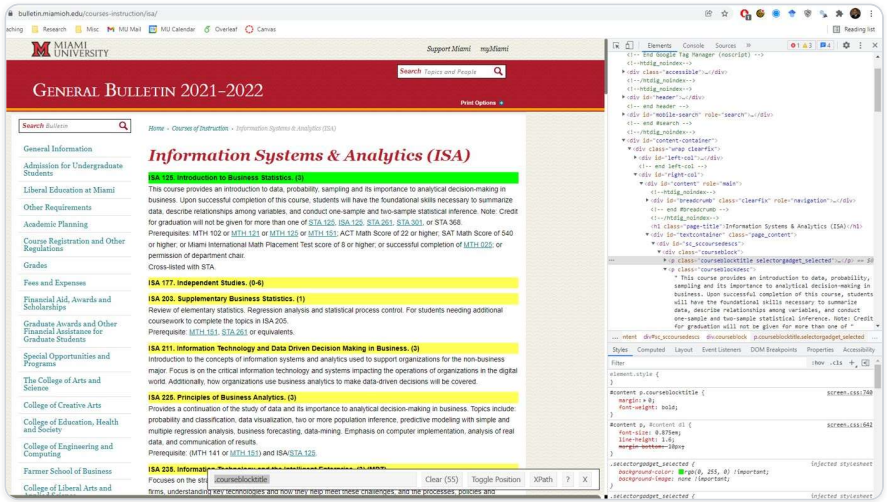
The Inspector (every browser)

Right-click an element on the page, then **Inspect**. The Elements panel opens at that element; its tag, class, and id are the parts of a selector.



SelectorGadget (Chrome extension)

Click an element: it turns green, similar ones turn yellow, and the bar at the bottom shows the selector and the count. Click a yellow one to exclude it.



Check the count. Whichever tool you use, compare the number of matches with what you see on the page before writing R code. Screenshots from an earlier edition of Miami's catalog; the current page uses the same tags.

Part 3: Scraping with rvest

The Scraping Workflow

1 Permission
robots.txt and the terms of service

2 Load
get the HTML into R

3 Select
find a CSS selector for the elements

4 Extract
table, text, or attribute

Do robots.txt and the site terms allow scraping this page?

Yes, both allow it

No, or I am not sure

Start over

Your code so far

Answer the questions on the left. Each answer adds the code for that step.

The rvest Process

1 Read

Download the page and parse the HTML into a tree R can search.

```
page = rvest::read_html(url)
```

Returns an HTML document:

```
{html_document}
```

→

2 Select

Keep every element that matches a CSS selector.

```
items = page |>
  rvest::html_elements("table.wikitable")
```

Returns the matching elements:

```
{xml_nodeset (9)}
```

→

3 Extract

Pull the data out of the elements, one of three ways:

`rvest::html_table()`	a list of data frames
`rvest::html_text2()`	a character vector of text
`rvest::html_attr("href")`	a character vector of attribute values

One rvest function per step. The selector from Part 2 is the only thing that changes from page to page, so most of the work happens in the browser, not in R.

rvest::html_elements() and rvest::html_element(). With an **s**: every match. Without it: the first match only. Wednesday uses the second one on pages with missing fields.

Some Sites Ask Who Is Sending the Request

```
wiki_url = "https://en.wikipedia.org/wiki/Miami_RedHawks_football"
```

A request that does not say who sent it

```
httr2::request(wiki_url) |>
  httr2::req_user_agent("") |>
  httr2::req_perform()
```

```
HTTP 403 Forbidden.
```

Wikipedia answered **403** to an empty User-Agent and **403** to one that only said `python-requests/2.31`, when these slides were built.

The same request, with a name and a way to reach you

```
wiki_page = httr2::request(wiki_url) |>
  httr2::req_user_agent("ISA 401 class scraper (your_email@miamioh.edu)") |>
  httr2::req_perform() |>
  httr2::resp_body_html()
```

Status **200**. `httr2::resp_body_html()` gives the same kind of document as `rvest::read_html()`, so steps 2 and 3 do not change.

User-Agent is a request header (Class 08) that names the program sending the request. Every browser sends one. Some sites refuse requests that leave it out or use a generic one, and the reply is **403 Forbidden**.

Wikimedia's User-Agent policy: *"Scripts should use an informative User-Agent string with contact information, or they may be blocked without notice."*

Choosing an Extraction Function

Three elements from the same Wikipedia page, and what each function returns for them.

I see rows and columns

```
<table class="wikitable">
  <tr><th>Season</th><th>Coach</th> ...
  <tr><td>1947</td> ...
```

↓

`rvest::html_table()`

Season	Bowl	Result
1947	Sun Bowl	W 13–12
1950	Salad Bowl	W 34–21

A data frame (one per table), with column names from the header row.

I need the text a reader sees

```
<h2 id="Bowl_games">Bowl games</h2>
```

↓

`rvest::html_text2()`

```
[1] "Bowl games"
```

The content, without tags, with spaces and line breaks cleaned up the way the browser shows them.

I need a value inside the tag

```
<a
href="https://en.wikipedia.org/wiki/1948_Sun_Bow
l" title="1948 Sun Bowl">Sun Bowl</a>
```

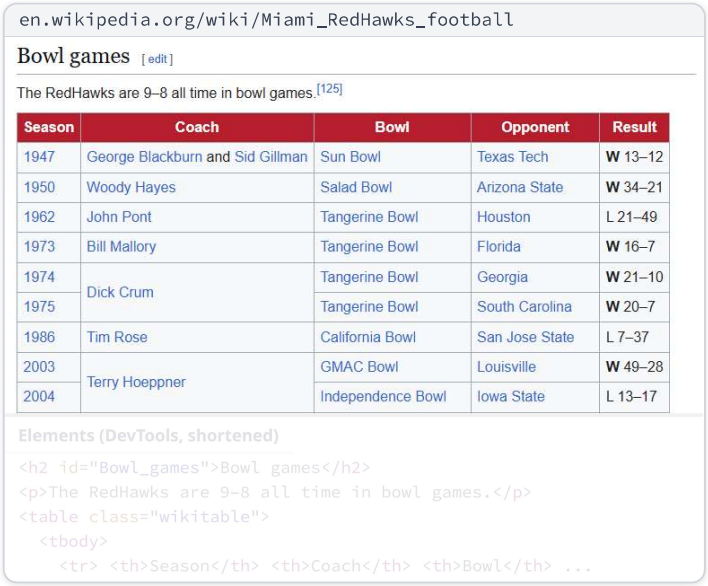
↓

`rvest::html_attr("href")`

```
[1] "https://en.wikipedia.org/wiki/1948_Sun_Bo
wl"
```

The attribute you name: `"href"` for links, `"src"` for images.

All the Tables on a Wikipedia Page



Selector:

```
wiki_url = "https://en.wikipedia.org/wiki/Miami_RedHawks_football"
wiki_page = httr2::request(wiki_url) |>
  httr2::req_user_agent("ISA 401 class scraper (your_email@miamioh.edu)") |>
  httr2::req_perform() |>
  httr2::resp_body_html()
wiki_tables = wiki_page |>
  rvest::html_elements("table.wikitable") |>
  rvest::html_table()
bowl_games = wiki_tables[[4]]
```

Output appears here when a line runs.

Goal. Every data table on this Wikipedia page, in R. We start in the browser, not in R.

Back | **Step** | **Play** | **Reset** | 0.5x | 1x | 2x

0 / 8

The Code and the Result

```r
wiki_url = "https://en.wikipedia.org/wiki/Miami_RedHawks_football"

robotstxt::paths_allowed(paths = "/wiki/Miami_RedHawks_football",
                         domain = "en.wikipedia.org", bot = "*")

wiki_page = httr2::request(wiki_url) |>
  httr2::req_user_agent("ISA 401 class scraper (your_email@miamioh
  httr2::req_perform() |>
  httr2::resp_body_html()

wiki_tables = wiki_page |>
  rvest::html_elements("table.wikitable") |>
  rvest::html_table()

length(wiki_tables)

bowl_games = wiki_tables[[4]]
bowl_games
```

wiki_tables: a list of 9 data frames

	Section	Size
[[1]]	Conference championships	24 x 5
[[2]]	Division championships	9 x 5
[[3]]	Head coaches	44 x 6
[[4]]	Bowl games	17 x 5
[[5]]	Cradle of Coaches	19 x 3
[[6]]	College Football Hall of Fame	8 x 5
[[7]]	Pro Football Hall of Fame	3 x 5
[[8]]	Retired numbers	4 x 6
[[9]]	Future non-conference opponents	5 x 9

bowl_games, rows 4 to 7 of 17 (Opponent not shown)

Season	Coach	Bowl	Result
1973	Bill Mallory	Tangerine Bowl	W 16–7
1974	Dick Crum	Tangerine Bowl	W 21–10
1975	Dick Crum	Tangerine Bowl	W 20–7
1986	Tim Rose	California Bowl	L 7–37

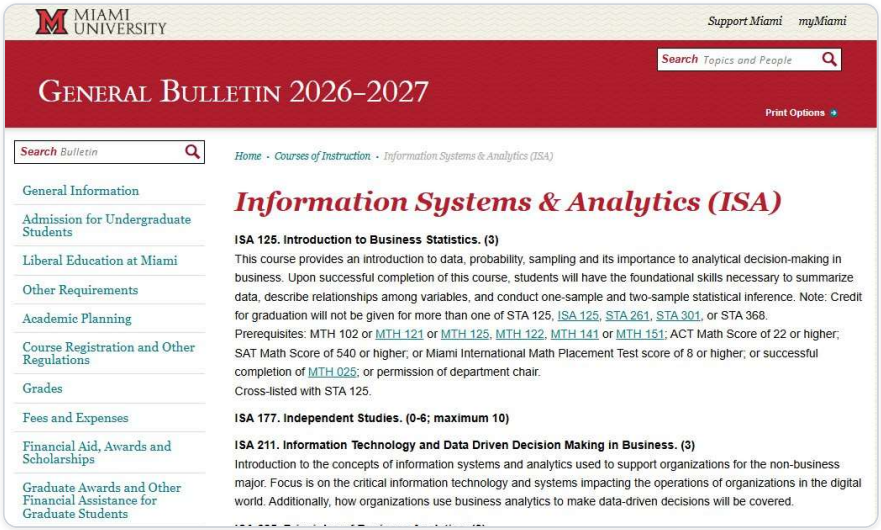
Dick Crum coached both the 1974 and 1975 games. On the page his name fills one merged cell; `rvest::html_table()` repeats it on each row.

Check the result against the page. The page says the RedHawks are 9-8 in bowl games. `sum(startsWith(bowl_games$Result, "W"))` gives **9** wins in 17 games.

Demo: Miami's ISA Course Catalog

Question | Titles | Descriptions | Data Frame | Discussion

Question. Build a data frame of every ISA course in Miami's 2026-2027 General Bulletin: one row per course, with its title and its description.



Page: bulletin.miamioh.edu/courses-instruction/isa/

Work through the workflow: permission, load, select, extract. Find both selectors in the browser first.

We write the code together. Type it in `class10.Rmd`.

Practice: The Fake Jobs Board

05:00

Task Answer Discussion

Two questions

1. Build a data frame with one row per job: title, company, location, date posted, and the address of its **Apply** link.

2. How many job titles contain the word "Python"?

Work in pairs, in `class10.Rmd`.

The page

realpython.github.io/fake-jobs

A static job board that Real Python built for scraping practice. The companies and jobs are made up.

Recap

Summary of Main Points

By now, you should be able to do the following:

- **Determine** when scraping is permitted: `robotstxt::paths_allowed()` for the path, then the site's terms of service. hiQ v. LinkedIn is the reminder that the terms count.
- **Parse** HTML with `rvest::read_html()`, or with `httr2` and a User-Agent when a site asks who is sending the request, then select elements with a CSS selector found in the Inspector.
- **Extract** tables with `rvest::html_table()`, text with `rvest::html_text2()`, and attribute values with `rvest::html_attr()`.

Next class (Wed, Sep 30): Web Scraping II: many pages with `purrr::map()`, polite delays between requests, and fields that are missing on some pages.

Before You Go

Join at **https://www.menti.com/al37nbpvnzpg**

🎯 Assignments 🎯

No new graded assignment today

Open assignments and their due dates are on the **Canvas Assignments page**. The practice questions on the right use today's three pages.

Before you leave: knit, then commit and push `markdowns/class10.Rmd` and `class10.html`. If it is not pushed, it is not done.

Reading, if you want more: R4DS 2e, Chapter 24: Web scraping and the rvest getting-started article.

Ungraded Practice Questions

1. **Wikipedia.** Which coach took Miami to the most bowl games? Use `bowl_games`.
2. **Wikipedia.** Get the link address of every bowl in the bowl games table.
3. **Catalog.** Which ISA courses carry exactly 3 credit hours? Count them.
4. **Fake Jobs.** Which state appears most often in the locations?
5. **Your pick.** Choose a site you use, find its robots.txt and its terms, and decide whether you may scrape it.

▶ **Answers**